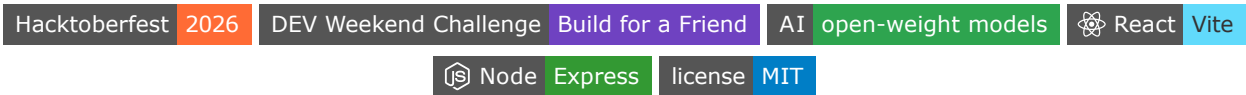


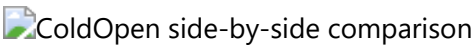
ColdOpen

Cold emails that sound like *you* wrote them.

Paste a few of your own messages. ColdOpen learns how you write and drafts three cold emails and three subject lines in your voice, then shows you what changed next to a generic AI draft.



[Live demo](#) · [Write-up on DEV](#) · [Report an issue](#)



Why this exists

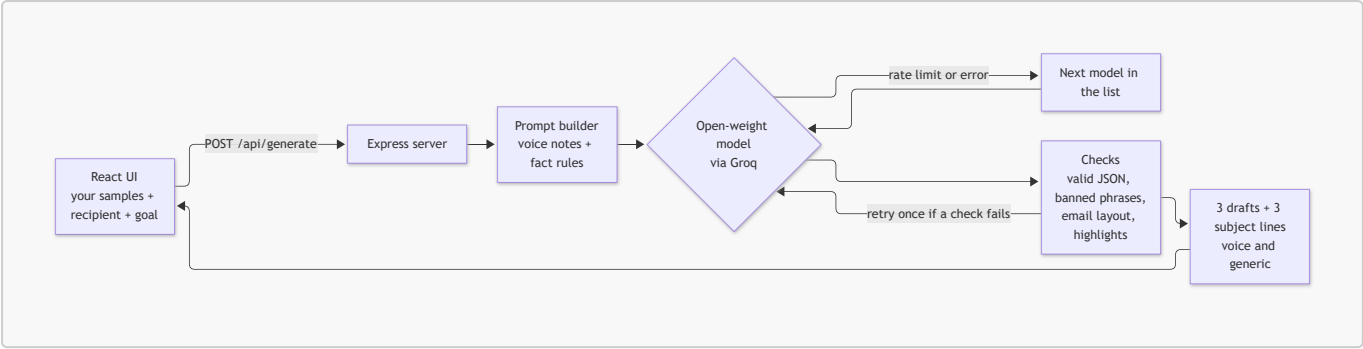
I built ColdOpen for my sister, a software engineer with 5+ years of experience. Writing to someone you don't know is hard, and the first message is the one most people never send. Generic AI email tools make it worse: the draft is polished, but it doesn't sound like you, so you don't send that one either.

ColdOpen starts from your own writing instead of a template.

What it does

Learns your voice	Reads 2-3 of your own messages and writes a short "voice learned" note (greeting, sign-off, sentence length, habits) that you can check.
Three drafts, three subject lines	Direct & Concise, Warm & Respectful, and Curiosity-First, each in your voice.
Voice vs. generic, side by side	Tick one box to see the same request written with no voice, so you can see what the voice step changed.
Highlights that explain themselves	Green marks phrasing that carries your voice, pink marks stock "template" phrases in the generic draft, and amber marks [placeholders] you must fill in before sending.
15 goals	Intro to an alum, referral, recruiter intro, resume feedback, mock interview, mentoring, follow-up, thank-you, reconnect, reference, switch roles, and more.
Doesn't make things up	The prompt only allows facts you typed in. Anything missing becomes a [bracketed placeholder].
One-click copy	Copies the selected subject line and the email body together.

How it works



1. **Voice notes first.** The model describes how you write before it drafts anything, and that description is shown in the UI.
2. **Fact rules.** The prompt forbids invented talks, projects, shared history or "I admire your work" lines. It may only use what you typed into About you, Your question and Anything specific.
3. **Guardrails around a small model.** The server validates the JSON, rejects stock clichés in the voice column (like "I hope you're doing well"), repairs one-line emails into a proper layout, and checks that every highlighted phrase really appears in the email.
4. **Model fallback.** Models are tried in order from `GROQ_MODELS`. If one is rate limited or unavailable, the next one is used.

Why open-weight models

- **Swappable.** The model list is one environment variable. When the models I planned on weren't available on my Groq account, switching was a config change, not a code change.
- **Inspectable.** Open weights mean I can see where the model stops and my code starts. Most of this project is guardrails around that gap.
- **Cheap to run.** The free tier covers a weekend of testing.
- **Portable.** The same prompts can point at a local runtime later.

An honest note on privacy: ColdOpen calls a *hosted* open-weight model, so your sample messages are sent to that provider to generate drafts. This app doesn't store them, but they do leave your machine. A local mode (for example Ollama) is on the roadmap and isn't built yet.

Tech stack

Layer	Choice
Frontend	React + Vite
Backend	Node.js + Express
AI	Open-weight models via the Groq API (<code>groq-sdk</code>), default <code>openai/gpt-oss-120b</code> with fallbacks
Hardening	Input length caps, rate limiting (10 requests/minute per IP), user text fenced as data inside the prompt, JSON validation with retries
Hosting	Render, as a single web service (Express serves the built React app)

Getting started

You need: Node.js 18+ and a free API key from console.groq.com.

```
git clone https://github.com/<your-username>/coldopen.git
cd coldopen

# 1. backend
cd server
cp .env.example .env          # then add your GROQ_API_KEY
npm install
npm run dev                    # http://localhost:5000

# 2. frontend (new terminal)
cd client
npm install
npm run dev                    # http://localhost:5173
```

Open <http://localhost:5173>. In development there is also a **Test data** bar above the form that fills it with one of 15 sample cases (one per goal) in one click. It is hidden in production builds.

Environment variables ([server/.env](#))

Variable	Required	Description
GROQ_API_KEY	yes	Your key from console.groq.com
GROQ_MODELS	no	Comma-separated models, tried in order. Default: openai/gpt-oss-120b , qwen/qwen3.8-27b , openai/gpt-oss-20b . Check which models your Groq account can use.
PORT	no	Server port (default 5000)
CLIENT_ORIGIN	no	Frontend URL for CORS in production

Deploy on Render

One web service is enough, because Express serves the built frontend.

Setting	Value
Build command	<code>cd client && npm install && npm run build && cd ../server && npm install</code>
Start command	<code>node server/index.js</code>
Environment	<code>GROQ_API_KEY, GROQ_MODELS</code>

API

POST </api/generate>

```
{
  "writingSamples": "2-3 of your own messages (80+ characters)",
  "recipientName": "Priya Sharma",
  "recipientRole": "Engineering Manager",
  "company": "Example Corp",
  "goal": "switch",
  "length": 100,
  "about": "Software engineer, 5+ years, backend work",
  "ask": "How does your team hire for senior roles?",
  "context": "We were in the same college tech club",
  "compare": true
}
```

Returns { `personalized`, `generic` }. Each has `voice_notes`, three `subject_lines` and three `variants` (`tone`, `body`, `highlights`). `generic` is `null` unless `compare` is `true`.

GET /api/health returns the configured model list.

Project structure

```
coldopen/
├── server/
│   ├── index.js          # API, prompt builder, guardrails, highlights
│   ├── package.json
│   └── .env.example
└── client/
    ├── src/
    │   ├── App.jsx       # form, results, highlighting
    │   ├── testData.json # 15 one-click test cases (dev only)
    │   └── styles.css
    └── vite.config.js    # proxies /api to the server in development
```

Limitations

- Language models sometimes add small details that weren't in your input. The fact rules and filters reduce this but don't remove it. **Read every draft before you send it.**
- The quality of the voice match depends on how much of your natural writing you paste in.
- Highlights are a guide, not a measurement. Green and pink come from the model's picks plus simple fallbacks (overlap with your samples, and a hand-written list of stock phrases). They are not an AI-text detector.
- Free-tier rate limits apply: a request with the generic comparison on makes two model calls.

Roadmap

- ☒ Local mode with Ollama, so samples never leave your laptop
- ☒ Gemma as an additional model, with a side-by-side voice comparison between models
- ☒ Remember your writing samples in the browser so you don't re-paste them

- ☒ "Make it shorter / more casual" per email
- ☐ Highlight facts in a draft that weren't in your input

Contributing

Issues and pull requests are welcome. For anything larger than a small fix, please open an issue first so we can agree on the approach. Please don't commit API keys: `.env` is ignored, and `.env.example` shows the variables you need.

Acknowledgements

Built over a weekend for the [Hacktoberfest 2026 Weekend Challenge: Build for a Friend](#). Thanks to the open-weight model teams and to the Groq and Render free tiers that made this free to build.

License

[MIT](#)